SPARXSTAR

Starmus Audio Recorder

Technical Specification & Standards Alignment Report

v1.0 — April 2026 // Starisian Technologies

Repository	sparxstar-starmus-audio
Purpose	Frontline audio acquisition — captures, consents, normalizes, and certifies audio artifacts for AiWA corpus ingestion
Stack	WordPress (latest stable), PHP 8.2+ strict, JavaScript (ES modules), GraphQL, TUS, Cloudflare edge, MariaDB, Redis, FFmpeg
Status	First-generation implementation — pre-Platform Integrity Map alignment
This Doc	Baseline tech spec + full standards-alignment audit against: Coding Standards Handbook v1.0, Platform Integrity Map v1.0, DVE Schema (digital-village-elder-schema)
Author	Starisian Technologies — Max Barrett
Date	April 2026

1. System Overview

1.1 Role in the SPARXSTAR Platform

Starmus Audio Recorder is the frontline acquisition layer of the SPARXSTAR platform. Its sole function is to capture audio, enforce contributor consent, normalize metadata to the DVE schema, and produce certified artifacts that graduate into the AiWA corpus. No data flows into AiWA until Starmus certifies it.

In platform terms, Starmus sits between the contributor-facing WordPress presentation layer and the governance pipeline. It is a data intake system — not a storage system, not a governance system, and not an AI system. Those concerns belong to Dheghom, Mḗh₁ns, and Sirus respectively.

1.2 Repository Structure

Layer	Path(s)	Responsibility
Entry	starmus-audio-recorder.phpsrc/StarmusAudioRecorder.php	Plugin bootstrap, dependency wiring, hook registration
API	src/api/	REST endpoints: upload-chunk, upload-fallback, upload-chunk-legacy, status
Core	src/core/	Submission handling, consent, post type registration, settings, asset loader
Data	src/data/	DAL, base DAL, prosody DAL, schema mapper, job repositories
Services	src/services/	Audio pipeline, FFmpeg, ID3, waveform, R2/S3 storage, post-processing, bandwidth detection
Frontend	src/frontend/	PHP: recorder UI, editor UI, re-recorder UI, consent UI, shortcode loader, prosody player
JavaScript	src/js/	Core engine, recorder, TUS upload, state store, UI, offline queue, sparxstar integration, transcript controller
Admin	src/admin/	Admin panel, SageMaker job queue, task manager, admin widgets
Integrations	src/integrations/	HuggingFace client, SageMaker client, app mode integration
Schema	acf-json/	ACF/SCF field definitions (DVE alignment target)
Templates	src/templates/	PHP templates: recorder, editor, re-recorder, consent form, recording detail, my-recordings list
i18n	src/i18n/, languages/	Internationalization, POT files
Tests	tests/	Unit (PHPUnit), Integration, E2E (Playwright), JS (metadata schema)

1.3 Bootstrap Contract

The JavaScript layer refuses to initialize without a page-type contract. Every page must inject the following object before any JS bundle runs:

```
window.STARMUS_BOOTSTRAP = {
  pageType: "recorder" | "rerecorder" | "editor",
```

```
  postId: number | null,
  restUrl: string,
  mode: "draft" | "development" | "production",
  canCommit: boolean,
  transcript: array | null,
  audioUrl: string | null
}
```

This bootstrap is authoritative for what the current page is permitted to do. JS modules check it before executing any governed action.

1.4 Data Flow

The recording lifecycle follows a strict one-way path:

Contributor → WordPress Page (Bootstrap injected) → UI Controller (Step 1 validation) → Recorder Engine (capture) → Submissions Handler (TUS or fallback) → WordPress REST API → Submission Handler PHP → DAL → MariaDB + Storage → Post-Processing Queue → AiWA corpus (after human/AI transcript approval)

The separation of concerns is enforced by module boundary rules: the UI controller never touches blobs; the recorder engine never touches the DOM; the submission handler never decides authority.

2. PHP Component Specifications

2.1 StarmusAudioDAL / StarmusBaseDAL

Attribute	Value
Class	StarmusAudioDAL extends StarmusBaseDAL implements IStarmusAudioDAL
Namespace	Starisian\Sparxstar\Starmus\data
Pattern	Repository pattern — all DB access mediated through typed interface methods
Strict Types	Declared — compliant
Return Types	Typed — int|WP_Error, bool, array patterns throughout
Transactions	Present in ConsentUI (START TRANSACTION / COMMIT / ROLLBACK)
Responsibility	create_audio_post, create_transcription_post, create_translation_post, create_attachment_from_file, metadata reads/writes, taxonomy assignment

2.2 StarmusSchemaMapper

Attribute	Value
Class	StarmusSchemaMapper (static methods)
Namespace	Starisian\Sparxstar\Starmus\data\mappers
Role	Translates legacy frontend field keys to canonical starmus_* schema keys. Acts as the primary DVE alignment boundary.
FIELD_MAP	~60 mapped fields covering: archival fields, session metadata, rights/credits, technical data, processing status, agreement, music engineering, transcription/translation
JSON Fields	Handled separately via ensure_json_string(): environment_data, waveform_json, transcription_json, recording_metadata
Complex Logic	dc_creator, user IDs (copyright_licensor, authorized_user_id), dates (dc_date_created, session_date), IP address normalization, taxonomy ID mapping

2.3 StarmusSubmissionHandler

Attribute	Value
Class	StarmusSubmissionHandler implements IStarmusSubmissionHandler (final)
Namespace	Starisian\Sparxstar\Starmus\core
Version	6.9.3-GOLDEN-MASTER
Upload Strategies	1. TUS chunked (primary), 2. Direct HTTP POST (fallback), 3. Base64 legacy (Tier C)
Dependencies	IStarmusAudioDAL (injected), StarmusSettings (injected), StarmusPostProcessingService
Validation	MIME type check (allowlist), filesize cap, extension check, path traversal protection

Workflow	File validate → secure move → WP attachment → audio post create/update → metadata via SchemaMapper → taxonomy assignment → post-processing trigger → temp cleanup
Allowed MIME	audio/webm, audio/ogg, audio/mpeg, audio/wav, audio/x-wav, audio/mp4

2.4 StarmusRESTHandler

Attribute	Value
Class	StarmusRESTHandler (final)
Namespace	Starisian\Sparxstar\Starmus\api
Endpoints	POST /wp-json/star/v1/upload-chunkPOST /wp-json/star/v1/upload-fallbackPOST /wp-json/star/v1/upload-chunk-legacyGET /wp-json/star/v1/status/{id}
Auth	current_user_can("upload_files") — WordPress capability gate on all write endpoints
Note	Uses WordPress-native capability checks. Sirus integration not yet present (see Section 4 — findings).

2.5 StarmusTusdHookHandler

Attribute	Value
Class	StarmusTusdHookHandler
Namespace	Starisian\Sparxstar\Starmus\includes
Role	Receives webhook callbacks from TUS daemon on upload completion events
Events	post-finish (triggers file processing), default (acknowledgment)
Security	x-starmus-secret header validation, path traversal protection, JSON-only communication
Endpoint	starmus/v1/hook namespace

2.6 StarmusAudioPipeline

Attribute	Value
Class	StarmusAudioPipeline (final)
Namespace	Starisian\Sparxstar\Starmus\services
Role	Orchestrates ID3 analysis + FFmpeg optimization + storage offload for uploaded audio
Steps	1. getID3 analysis, 2. Write Starmus metadata tags to original, 3. Generate web-optimized versions (FFmpeg), 3b. Offload to storage, 4. Waveform generation
Warning	Section 3b contains unresolved overlap between manual FFmpeg calls and StarmusR2DirectService::processAfricaAudio. Reconciliation required (see findings).

2.7 StarmusR2DirectService

Attribute	Value
Class	StarmusR2DirectService (final)
Namespace	Starisian\Sparxstar\Starmus\services
Role	S3-compatible storage operations for processed audio assets. Supports Cloudflare R2 (primary) and AWS S3 (backup).
SDK	Uses Aws\S3\S3Client directly — violates provider abstraction rule (see findings)
Provider Select	STARMUS_STORAGE_PROVIDER constant: "r2" (default) or "aws"

3. JavaScript Module Specifications

3.1 Module Inventory

Module	Role	Compliance Notes
starmus-core.js	Config, tier detection, TUS client init, global state	TUS chunk size defaults 512KB — compliant
starmus-recorder.js	MediaRecorder lifecycle, gain analysis, meter, blob normalization	sampleRate=16000, channelCount=1, bitrate cap=32000 — compliant. Duration limit deferred to bootstrap tier, not enforced at capture layer — gap (see findings).
starmus-tus.js	TUS chunked upload with resume support, IndexedDB queue	chunkSize from config (512KB) — compliant. Checksum handling: verify in findings.
starmus-state-store.js	Centralized state management, event-driven updates	Review for mutable global state
starmus-ui.js	DOM manipulation, visual feedback, meter rendering	Separation maintained — no blob access
starmus-offline.js	IndexedDB FIFO queue for offline upload queuing	Required by Coding Standards §3.5 — present
starmus-hooks.js	CommandBus — internal event/command system	Clean separation
starmus-integrator.js	Wires all modules into coherent system	Review for initialization guard on STARMUS_BOOTSTRAP
starmus-audio-editor.js	Peaks.js waveform, region annotation, transcript sync	Loads conditionally on pageType=editor only — compliant
starmus-transcript-controller.js	Transcript scroll sync, cue events	Review for sensor/event throttling compliance
starmus-metadata-auto.js	Automatic metadata extraction from capture context	Verify no client-supplied timestamps used for ordering
starmus-sparxstar-integration.js	SPARXSTAR platform integration bridge	Review — verify Sirus authority resolution path

3.2 Audio Capture Constraints

Parameter	Standard Limit	Implementation	Status
Sample Rate	16,000 Hz max	settings.sampleRate || 16000	COMPLIANT
Channels	1 (mono)	settings.channels || 1	COMPLIANT
Bitrate	32 kbps hard cap	settings.bitrate || 32000	COMPLIANT
Format	Opus or AAC-LC	Configured in MediaRecorder mimeType — verify per build	VERIFY

Max Duration	120s production180s development300s draft	Not enforced at recorder module level — depends on tier bootstrap settings	GAP — see findings

4. Standards Alignment Audit

This section documents all identified gaps, violations, and alignment requirements against: the SPARXSTAR Coding Standards Handbook v1.0 (CS), the Platform Integrity Map v1.0 (PIM), and the DVE Schema canonical reference.

4.1 Finding Summary

ID	Severity	Location	Description	Required Fix
F-01	**CRITICAL**	All REST endpoints, all governed actions	Sirus (cross-repo authority layer) is entirely absent. No Sirus::resolveContext() or Sirus::resolveAuthority() call exists anywhere in the codebase. All authority decisions are made locally using WordPress capabilities (current_user_can) or raw user ID checks.	All governed actions must call Sirus before execution. This is a hard CI failure condition under CS §1.3 and a Platform Integrity Map invariant. This is the highest-priority alignment item.
F-02	**CRITICAL**	src/services/StarmusR2DirectService.php	Direct use of Aws\S3\S3Client in application code without an abstraction layer. The AWS SDK is instantiated directly in the service class — a violation of CS §0.7 (Infrastructure Provider Selection).	Create a StorageClient interface. Implement R2Adapter and S3Adapter. Inject the interface rather than instantiating the SDK directly. The service should never reference a provider SDK by name in application code.
F-03	**HIGH**	src/api/StarmusRESTHandler.php, src/core/StarmusSubmissionHandler.php	No Sirus consent verification on governed actions. CS §2.8 requires sparxstar_has_consent() before any governed action. Current implementation checks WordPress capabilities only — consent scope is not verified at the action layer.	Add Sirus consent verification to all governed REST endpoints and submission handler actions. Do not assume consent from a prior consent post existence — verify at execution time.
F-04	**HIGH**	src/js/starmus-recorder.js	Maximum recording duration is not enforced at the recorder module level. CS §4.1 specifies 120s (production), 180s (development), 300s (draft). Duration is referenced in bootstrap tier settings but not enforced at capture. A long recording will not be stopped.	Add a timer in RecorderEngine that hard-stops recording at the mode-appropriate max duration. Enforce at the module boundary — not only at the UI level.
F-05	**HIGH**	src/services/StarmusAudioPipeline.php §3b	Unresolved overlap between manual FFmpeg web version generation and StarmusR2DirectService::processAfricaAudio. Both paths produce web-optimized versions. It is unclear which path runs in production and whether duplicate processing occurs.	Designate one authoritative path. If R2DirectService::processAfricaAudio is the production path, remove the duplicate FFmpeg loop in StarmusAudioPipeline. Document the decision as a locked architectural choice.

F-06	**HIGH**	src/data/StarmusJobSearchRepository.php	SELECT * used in two queries (job_id lookup and paginated list). CS §2.4 prohibits SELECT * in production queries.	Replace SELECT * with explicit column lists matching the StarmusJob schema. Ensure LIMIT is present on all paginated queries (it appears to be, but verify the un-paginated lookup path).
F-07	**MEDIUM**	src/data/mappers/StarmusSchemaMapper.php	PASSTHROUGH_ALLOWLIST and FIELD_MAP coexist. The passthrough list contains legacy keys (starmus_global_uuid, starmus_stable_uri etc.) that are also present as values in FIELD_MAP. This creates potential for double-mapping and stale passthrough fields that bypass the canonical field map.	Consolidate to FIELD_MAP only. Remove PASSTHROUGH_ALLOWLIST entirely. Any field that should pass through should be explicitly mapped old_key => same_key in FIELD_MAP so the mapping is visible and auditable.
F-08	**MEDIUM**	src/data/mappers/StarmusSchemaMapper.php line ~200	date('Y-m-d H:i:s') used for agreement_datetime — client-side execution of date generation. CS §12.2 requires server-side time for all ordering and session decisions. PHP date() is server-side but does not use gmdate() — local timezone may affect output.	Replace date() with gmdate() consistently. All timestamps written to DB must be UTC. Add a constant timezone assertion at plugin bootstrap.
F-09	**MEDIUM**	src/integrations/StarmusSageMakerClient.php, StarmusHuggingFaceClient.php	Named AI provider integrations are hardcoded in application code. Under PIM Commercial Plugin Hostility Assumption (Section 0), no guarantee may depend on third-party service cooperation. These integrations have no fallback or abstraction layer.	Create an AIProcessingClient interface. Implement SageMakerAdapter and HuggingFaceAdapter. Make the active adapter a configuration choice, not a compile-time dependency. Document fallback behavior when the AI service is unavailable.
F-10	**MEDIUM**	src/js/starmus-tus.js	TUS chunk checksum per-chunk verification status unclear. CS §5.1 requires checksum verification per chunk. The chunk size of 512KB is compliant. UUID assignment per upload is present via TUS protocol. Verify checksum header is being sent and validated.	Audit TUS client configuration for checksum extension. Ensure the server-side TusdHookHandler validates checksums. Add CI test asserting checksum header presence on chunk upload.
F-11	**MEDIUM**	Multiple templates	Several templates use current_user_can("administrator"), current_user_can("manage_options"), current_user_can("super_admin") as authorization gates. These are local permission checks without Sirus delegation — a CS §1.3 violation.	All permission decisions must be delegated to Sirus. Template-level capability checks are acceptable for UI rendering decisions (show/hide) only — they must never gate data writes or governed actions.

F-12	**LOW**	src/data/StarmusJobSearchRepository.php	Uses $this->db->query("START TRANSACTION") style raw query strings in ConsentUI (StarmusConsentUI.php line 150). While functional, this bypasses WPDB prepared statement pattern.	Transaction control (START TRANSACTION, COMMIT, ROLLBACK) is acceptable as raw queries since they carry no user-supplied data. Document this as an intentional exception in the codebase comment.
F-13	**LOW**	src/StarmusAudioRecorder.php	runtimeErrors check uses current_user_can("manage_options") as an error visibility gate — a local capability check. Not a governed action, so Sirus is not required here, but should be documented as an intentional UI-only check.	Add inline comment: "UI display gate only — not a governed action. Sirus delegation not required."
F-14	**LOW**	src/js/starmus-recorder.js LanguageSignalAnalyzer	LanguageSignalAnalyzer uses a 20-second active sensor window (maxDuration = 20000ms). CS §3.1 specifies sensor active window of 5000ms then auto-disable.	Reduce LanguageSignalAnalyzer active window to 5000ms maximum, or classify the speech recognition probe as a non-sensor governed action and document the exception with architectural justification.

5. DVE Schema Alignment

This section compares the Starmus field space against the canonical DVE (Digital Village Elder) schema. The DVE schema is the authoritative reference — sparxstar-digital-village-elder-schema repo. Field names in Starmus must match DVE field names exactly where they correspond to DVE-governed data.

5.1 Current SCF Schema (acf-json/)

The current ACF/SCF JSON schema in starmus-audio-recorder-scf.json defines fields under the aiwa_ prefix for dictionary/linguistic data. These fields are distinct from the starmus_ prefixed fields managed by StarmusSchemaMapper — they serve different use cases.

Field Name	Type	DVE Status
aiwa_translation	text	AiWA linguistic field — not a DVE core field. Retain as-is.
aiwa_translation_english	text	AiWA linguistic field — retain.
aiwa_translation_french	text	AiWA linguistic field — retain.
aiwa_part_of_speech	select	AiWA linguistic field — retain.
aiwa_search_string_english	text	AiWA — retain.
aiwa_search_string_french	text	AiWA — retain.
aiwa_rating_average	number	AiWA — retain. NOTE: rating average should be server-computed, not stored as editable field.
aiwa_ipa_pronunciation	text	AiWA — retain.
aiwa_audio_file	file	ALIGNMENT REQUIRED — see 5.2 below.
aiwa_origin	text?	AiWA provenance — verify against DVE asset_provenance field.
aiwa_word_photo	file	AiWA — retain.
aiwa_example_sentences	text	AiWA — retain.
aiwa_extract	text	AiWA — verify scope vs. DVE dc_description.
aiwa_synonyms	text	AiWA — retain.
aiwa_rating	?	Ambiguous — different from aiwa_rating_average? Clarify or remove.
aiwa_qc_status	?	ALIGNMENT REQUIRED — map to DVE qa_review or starmus_qa_review field.
aiwa_qc_notes	?	ALIGNMENT REQUIRED — map to DVE qa_notes or equivalent.

5.2 SchemaMapper Field Alignment

The following fields in StarmusSchemaMapper require DVE schema validation. Where a starmus_ prefixed field corresponds to a DVE canonical field, the names must match exactly.

starmus_ Field	DVE Canonical Field	Status

starmus_global_uuid	asset_global_uuid	VERIFY — confirm DVE uses asset_global_uuid
starmus_stable_uri	asset_stable_uri	VERIFY
starmus_linked_data_uri	linked_data_uri	LIKELY ALIGNED
starmus_rights_type	dc_rights_type	LIKELY ALIGNED
starmus_rights_use	dc_rights	VERIFY — mapped from two source keys (dc_rights AND usage_restrictions_rights), potential conflict
starmus_data_sensitivity	data_sensitivity_level	VERIFY field name matches DVE exactly
starmus_anon_status	anonymization_status	VERIFY — consider DVE may use full name
starmus_consent_scope	consent_scope	LIKELY ALIGNED
starmus_transcription_text	transcription	VERIFY — DVE may use transcription_text or transcription
starmus_transcription_json	transcription_json	LIKELY ALIGNED
starmus_translation_text	translation	VERIFY
starmus_waveform_json	waveform_json	LIKELY ALIGNED
starmus_agree_ip	contributor_ip	RENAME — three source keys all mapping to starmus_agree_ip creates field aliasing. DVE canonical should be single source of truth.
starmus_agree_ua	contributor_user_agent	VERIFY
starmus_qa_review	qa_review	VERIFY — must align with aiwa_qc_status above
starmus_session_location	location	VERIFY — DVE may use session_location
starmus_session_gps	gps_coordinates	VERIFY
starmus_cloud_object_uri	cloud_object_uri	LIKELY ALIGNED
starmus_archival_wav	archival_wav	LIKELY ALIGNED
starmus_mastered_mp3	mastered_mp3	VERIFY — DVE may use processed_audio_uri pattern

5.3 DVE Alignment Action Items

#	Action	Priority	Owner
D-01	Pull canonical DVE field list from sparxstar-digital-village-elder-schema and do a field-by-field comparison against FIELD_MAP	HIGH	Before any new fields are added to Starmus
D-02	Resolve aiwa_qc_status / aiwa_qc_notes →	HIGH	Affects QC workflow

	starmus_qa_review mapping. The SCF JSON and SchemaMapper must agree on the canonical name		
D-03	Clarify aiwa_rating vs aiwa_rating_average — one field or two? If one, remove the duplicate	MEDIUM	Schema cleanliness
D-04	Resolve starmus_agree_ip triple-alias (ip_address, submission_ip, contributor_ip → all to same field). Pick one canonical source key	MEDIUM	SchemaMapper consolidation
D-05	Verify starmus_rights_use is not double-written (both dc_rights and usage_restrictions_rights map to it). If both sources can arrive, last-write wins — document or fix	MEDIUM	Data integrity
D-06	aiwa_audio_file field: verify this is the same asset reference as starmus_cloud_object_uri or starmus_archival_wav — or is it a separate ACF file field?	MEDIUM	Asset linkage clarity
D-07	Confirm all DVE timestamp fields use UTC (gmdate) not local time date(). Audit SchemaMapper date handling lines	MEDIUM	Timestamp integrity (ties to F-08)

6. Platform Integrity Map Alignment

The Platform Integrity Map v1.0 establishes 12 enforcement rules and 5 platform invariants that apply to all SPARXSTAR repos. This section audits Starmus against each.

6.1 Platform Invariants

Invariant	Description	Starmus Status
Invariant 1	No guarantee may depend on third-party plugin cooperation (Commercial Plugin Hostility Assumption)	PARTIAL FAIL — AWS SDK and SageMaker/HuggingFace clients are direct dependencies without abstraction. See F-02, F-09.
Invariant 2	Schema loading boundaries per component	COMPLIANT — SchemaMapper defines clear field boundary. ACF fields defined in acf-json/.
Invariant 3	QUARANTINE is a success state	NOT APPLICABLE at this layer — Starmus sends to Dheghom/Mḗh₁ns which implement quarantine. Verify handoff triggers quarantine correctly.
Invariant 4	"Data is never silently destroyed"	COMPLIANT — StarmusSubmissionHandler implements rollback on failure. TUS supports resume. Dead-letter handling: verify (CS §12.3).
Invariant 5	Draft encryption is mandatory	NOT VERIFIED — No encryption layer identified in current codebase. Audio stored as files. Verify if Cloudflare R2 bucket-level encryption satisfies this invariant or if application-level encryption is required.

6.2 Governance Scope

Starmus operates in Governed/Strict scope for all audio submission actions (consent, upload, metadata save). It operates in Contextual/Light scope for audio playback and recording list views. Ungoverned/Open scope applies to public-facing archive views only.

Current implementation does not enforce scope tiers explicitly — all actions are gated at WordPress capability level. Sirus integration (F-01) is required to enforce scope correctly.

6.3 Standalone Operational Rule

Each SPARXSTAR component must be independently operable. Starmus must function if Sirus is temporarily degraded — but in degraded mode, it must fail closed: no governed action proceeds without a valid Sirus context. This is currently untested because Sirus is not integrated.

6.4 WordPress as Presentation Layer

The Coding Standards Handbook and Platform Integrity Map designate WordPress as presentation layer only. Sovereignty is enforced through mu-plugin architecture and a Cloudflare edge layer. Starmus correctly treats WordPress as a rendering and routing surface — data validation, authorization, and storage are handled in the service and DAL layers.

Gap: The mu-plugin boundary is not yet defined for Starmus. Verify that Starmus is loaded as a mu-plugin or through a sovereign loader, not as a standard plugin that can be deactivated without governance consequences.

7. CI Enforcement Gaps

The following CI enforcement rules from Coding Standards §13 are not currently verifiable from the codebase audit. Each requires a corresponding CI check to be added.

CS Rule	Requirement	Current State
§1.5	Governed action exists without preceding Sirus call → FAIL	No CI check — Sirus not integrated (F-01)
§1.5	Local permission check without Sirus delegation → FAIL	No CI check — multiple local permission checks exist
§2.4	SELECT * in any query → FAIL	No CI check — SELECT * found in StarmusJobSearchRepository (F-06)
§4.1	audio sampleRate > 16000 → FAIL	Default is 16000 — verify CI enforces this as a build-time check
§4.1	audio channels > 1 → FAIL	Default is 1 — verify build-time enforcement
§4.1	audio bitrate > 32000 → FAIL	Default is 32000 — verify build-time enforcement
§5.1	upload chunk > 512 KB → FAIL	Default is 512KB — verify server-side enforcement, not just client-side config
§5.1	upload without chunk checksum → FAIL	Checksum status unclear (F-10) — no CI check confirmed
§5.1	upload without UUID → FAIL	TUS assigns UUID — verify server validates presence
§12.3	job silently discarded after max retries without dead-letter entry → FAIL	Dead-letter queue not confirmed in async job processing — verify SageMaker job failure path
§0.7	direct provider-specific API call without abstraction layer → FAIL	No CI check — Aws\S3\S3Client used directly (F-02)
§13.6	CSS bundle exceeds 50 KB → FAIL	Four CSS files present — verify combined bundle size
§13.2	JS bundle exceeds 150 KB gzipped → FAIL	starmus-audio-recorder-script.bundle.min.js — verify gzipped size

8. Remediation Plan

Findings are sequenced by dependency. Critical findings block all other work where noted.

Phase 1 — Critical Blockers

These items must be resolved before any new feature work proceeds.

Finding	Work Item	Blocker For
F-01	Implement Sirus integration: add Sirus::resolveContext() and Sirus::resolveAuthority() calls to StarmusRESTHandler, StarmusSubmissionHandler, and all governed resolver paths. Fail closed on null context.	F-03, F-11, Invariant enforcement
F-02	Create StorageInterface. Implement R2StorageAdapter and S3StorageAdapter wrapping Aws\S3\S3Client. Inject interface into StarmusR2DirectService. Remove direct SDK reference from application code.	§0.7 CI compliance, Invariant 1

Phase 2 — High Priority

Finding	Work Item
F-03	After F-01 complete: add Sirus consent verification to all governed endpoints. Do not assume consent from WordPress meta — verify scope at execution time.
F-04	Add hard duration cap to RecorderEngine. Use STARMUS_BOOTSTRAP.mode to select 120/180/300 second limit. Auto-stop and surface a user-visible reason.
F-05	Designate single authoritative processing path in StarmusAudioPipeline. Remove the duplicate FFmpeg web-version loop if R2DirectService::processAfricaAudio is the production path.
F-06	Replace SELECT * in StarmusJobSearchRepository with explicit column lists. Add LIMIT assertion to CI.
F-09	Create AIProcessingClient interface. Implement SageMakerAdapter and HuggingFaceAdapter. Document fallback behavior for AI service unavailability.

Phase 3 — Schema & Alignment

Finding	Work Item
F-07	Consolidate PASSTHROUGH_ALLOWLIST into FIELD_MAP. Eliminate dual-mapping path in SchemaMapper.
F-08	Replace all date() calls with gmdate(). Assert UTC timezone at plugin bootstrap. Audit all DB timestamp writes.
D-01	Pull canonical DVE schema field list. Produce field-by-field diff against FIELD_MAP. Update any mismatched field names.
D-02	Resolve aiwa_qc_status / starmus_qa_review mapping. Update ACF JSON and SchemaMapper to agree.

D-03–D-07	Resolve remaining DVE alignment items per Section 5.3.

Phase 4 — Hardening

Finding	Work Item
F-10	Audit TUS client for checksum extension configuration. Verify server-side validation. Add CI test.
F-11	Audit all templates — ensure no data writes are gated on local capability checks. UI display gates are acceptable; action gates are not.
F-14	Reduce LanguageSignalAnalyzer sensor window to 5000ms or document exception.
F-12, F-13	Add inline documentation for intentional exceptions (transaction control, UI-only capability gate).
CI	Add CI checks for all items in Section 7. Verify JS and CSS bundle sizes. Confirm audio parameter enforcement is build-time, not runtime-only.

9. Architectural Boundaries — What This Repo Is Not

These are locked architectural decisions for Starmus. They define what must not be added to this repository — concerns that belong to other platform components.

Capability	Belongs To	Starmus Role
Governance enforcement	Mḗhiņs	Starmus submits intake data. Mḗhiņs decides what passes.
Authority resolution	Sirus	Starmus calls Sirus. Starmus never decides authority.
Asset sovereignty storage	Dheghom	Starmus produces artifacts and hands them off. Does not own the vault.
Edge agreement	Helios	Starmus sits behind Helios. Does not implement edge trust logic.
Corpus management	AiWA pipeline	Starmus certifies artifacts for graduation. AiWA ingests them.
Transcription AI	External service via abstraction	Starmus stores transcription text/JSON. Does not run the AI.
Schema canonical reference	digital-village-elder-schema	Starmus field names must conform to DVE. Starmus does not own the schema.

Engineering Statement

Starmus Audio Recorder is a first-generation acquisition system built before platform invariants were fully locked.

It is structurally sound. The separation of concerns is correct. The data model is coherent.

What it needs is authority integration (Sirus), provider abstraction, and schema alignment. These are additive changes, not rewrites.

Version: 1.0 | Starisian Technologies | April 2026

Repo: sparxstar-starmus-audio | Standard References: Coding Standards Handbook v1.0, Platform Integrity Map v1.0, DVE Schema